

Introduction to 8086 Assembly Language

Assembly Language Programming

Program Statements

- Program consist of statement, one per line.
- Each statement is either an **instruction**, which the assembler translate into machine code, or **assembler directive**, which instructs the assembler to perform some spesific task, such as allocating memory space for a variable or creating a procedure.
- Both instructions and directives have up to four fields:

name operation operand(s) comment

- At least one blank or tab character must separate the fields

Program Statements

- An example of an instruction is

START: MOV CX,5 ; initialize counter

name operation operand(s) comment

- The name field consists of the label **START:**
- The operation is **MOV**, the operands are **CX** and **5**
- And the comment is **; initialize counter**

- An example of an assembler directive is

MAIN PROC

name operation operand(s) comment

- **MAIN** is the name, and the operation field contains **PROC**
- This particular directive creates a procedure called **MAIN**

Program Statements

 **name operation operand(s) comment**

- A Name field identifies a label, variable, or symbol. It may contain any of the following character :
A,B.....Z ; a,b.....z ; 0,1.....9 ; ? ;
_ (underline) ; **@** ; **\$** ; **.** (period)
- Only the first 31 characters are recognized
- There is no distinction between uppercase and lower case letters.
- The first character may not be a digit
- If it is used, the period (**.**) may be used only as the first character.
- A programmer-chosen name may not be the same as an assembler reserved word.

Program Statements

A Keyword, or reserved word, always have some predefined meaning to the assembler.

It may be **an instruction (MOV, ADD)**, or it may be **an assembler directive. (PROC, TITLE, END)**

Program Statements

name operation operand(s) comment



- Operation field is a predefined or reserved word
 - mnemonic - symbolic operation code.
 - The assembler translates a symbolic opcode into a machine language opcode.
 - Opcode symbols often describe the operation's function; for example, MOV, ADD, SUB
 - assembler directive - pseudo-operation code.
 - In an assembler directive, the operation field contains a pseudo-operation code (pseudo-op)
 - Pseudo-op are not translated into machine code; for example the PROC pseudo-op is used to create a procedure

Program Statements

name operation operand(s) comment



- An operand field specifies the data that are to be acted on by the operation.
- An instruction may have zero, one, or two operands.

For example:

- **NOP** No operands; does nothing
- **INC AX** one operand; adds 1 to the contents of AX
- **ADD WORD1,2** two operands; adds 2 to the contents of memory
WORD1

Program Statements

name operation operand(s) comment



- The comment field is used by the programmer to say something about what the statement does.
- A semicolon marks the beginning of this field, and the assembler ignores anything typed after semicolon.
- Comments are optional, but because assembly language is low level, it is almost impossible to understand an assembly language program without comments.

Program Data and Storage

- Pseudo-ops to define data or reserve storage
 - DB - byte(s)
 - DW - word(s)
 - DD - doubleword(s)
 - DQ - quadword(s)
 - DT - tenbyte(s)
- These directives require one or more operands
 - define memory contents
 - specify amount of storage to reserve for run-time data

Defining Data

- Numeric data values
 - 100 - decimal
 - 100B - binary
 - 100H - hexadecimal
 - '100' - ASCII
 - "100" - ASCII
- Use the appropriate DEFINE directive (byte, word, etc.)
- A list of values may be used - the following creates 4 consecutive words
`DW 40CH,10B,-13,0`
- A ? represents an uninitialized storage location
`DB 255,?, -128, 'x'`

Naming Storage Locations

- Names can be associated with storage locations

ANum DB -4

DW 17

ONE

UNO DW 1

X DD ?

- These names are called variables

- ANum refers to a byte storage location, initialized to FCh
- The next word has no associated name
- ONE and UNO refer to the same word
- X is an uninitialized doubleword

- Multiple definitions can be abbreviated

Example:

```
message DB 'B'  
        DB 'y'  
        DB 'e'  
        DB 0DH  
        DB 0AH
```

can be written as

```
message DB 'B','y','e',0DH,0AH
```

- More compactly as

```
message DB 'Bye',0DH,0AH
```

Arrays

- Any consecutive storage locations of the same size can be called an array

```
X DW 40CH,10B,-13,0
```

```
Y DB 'This is an array'
```

```
Z DD -109236, FFFFFFFFH, -1, 100B
```

- Components of X are at X, X+2, X+4, X+6
- Components of Y are at Y, Y+1, ..., Y+15
- Components of Z are at Z, Z+4, Z+8, Z+12

DUP

- Allows a sequence of storage locations to be defined or reserved
- Only used as an operand of a define directive

DB 40 DUP (?) ; 40 words, uninitialized

DW 10h DUP (0) ; 16 words, initialized as 0

Table1 DW 10 DUP (?) ; 10 words, uninitialized

**message DB 3 DUP ('Baby') ; 12 bytes, initialized
; as BabyBabyBaby**

**Name1 DB 30 DUP ('?') ; 30 bytes, each
; initialized to ?**

DUP

- The DUP directive may also be nested

Example

```
stars DB 4 DUP(3 DUP (*),2 DUP (?),5 DUP (!))
```

Reserves 40-bytes space and initializes it as

```
***??!!!!***??!!!!***??!!!!***??!!!!
```

```
matrix DW 10 DUP (5 DUP (0))
```

defines a 10X5 matrix and initializes its elements to zero.

This declaration can also be done by

```
matrix DW 50 DUP (0)
```

Word Storage

- Word, doubleword, and quadword data are stored in reverse byte order (in memory)

Directive	Bytes in Storage
DW 256	00 01
DD 1234567H	67 45 23 01
DQ 10	0A 00 00 00 00 00 00 00
X DW 35DAh	DA 35

Low byte of X is at X, high byte of X is at X+1

Word Storage

Symbol Table

- * Assembler builds a symbol table so we can refer to the allocated storage space by the associated label

Example

.DATA			name	offset
value	DW	0	value	0
sum	DD	0	sum	2
marks	DW	10 DUP (?)	marks	6
message	DB	'The grade is:', 0	message	26
char1	DB	?	char1	40

Named Constants

- Symbolic names associated with storage locations represent addresses
- Named constants are symbols created to represent specific values determined by an expression
- Named constants can be numeric or string
- Some named constants can be redefined
- No storage is allocated for these values

Equal Sign Directive

- name = expression
 - expression must be numeric
 - these symbols may be redefined at any time

```
maxint = 7FFFh
```

```
count = 1
```

```
DW count
```

```
count = count * 2
```

```
DW count
```

EQU Directive

- name EQU expression
 - expression can be string or numeric
 - Use < and > to specify a string EQU
 - these symbols cannot be redefined later in the program

`sample EQU 7Fh`

`aString EQU <1.234>`

`message EQU <This is a message>`

Data Transfer Instructions

- **MOV *target, source***
 - reg, reg
 - mem, reg
 - reg, mem
 - mem, imm
 - reg, imm
- Sizes of both operands must be the same
- reg can be any non-segment register except IP cannot be the target register
- MOV's between a segment register and memory or a 16-bit register are possible

Sample MOV Instructions

```
b db 4Fh  
w dw 2048
```

```
mov bl,dh  
mov ax,w  
mov ch,b  
mov al,255  
mov w,-100  
mov b,0
```

- When a variable is created with a define directive, it is assigned a default size attribute (byte, word, etc)
- You can assign a size attribute using LABEL

```
LoByte LABEL BYTE
```

```
aWord DW 97F2h
```

Addresses with Displacements

```
b db 4Fh, 20h, 3Ch
w dw 2048, -100, 0
```

```
mov bx, w+2
```

```
mov b+1, ah
```

```
mov ah, b+5
```

```
mov dx, w-3
```

- Type checking is still in effect

- The assembler computes an address based on the expression
- *NOTE: These are address computations done at assembly time*
MOV ax, b-1
will not subtract 1 from the value stored at b

eXCHanGe

- XCHG *target, source*
 - reg, reg
 - reg, mem
 - mem, reg
- MOV and XCHG cannot perform memory to memory moves
- This provides an efficient means to swap the operands
 - No temporary storage is needed
 - Sorting often requires this type of operation
 - This works only with the general registers

Arithmetic Instructions

ADD *dest, source*

SUB *dest, source*

INC *dest*

DEC *dest*

NEG *dest*

- *Operands must be of the same size*

- *source* can be a general register, memory location, or constant
- *dest* can be a register or memory location
 - except operands cannot both be memory

Program Segment Structure

- Data Segments
 - Storage for variables
 - Variable addresses are computed as offsets from start of this segment
- Code Segment
 - contains executable instructions
- Stack Segment
 - used to set aside storage for the stack
 - Stack addresses are computed as offsets into this segment
- Segment directives
 - `.data`
 - `.code`
 - `.stack size`

Memory Models

- `.Model memory_model`
 - tiny: code+data $\leq$ 64K (.com program)
 - small: code $\leq$ 64K, data $\leq$ 64K, one of each
 - medium: data $\leq$ 64K, one data segment
 - compact: code $\leq$ 64K, one code segment
 - large: multiple code and data segments
 - huge: allows individual arrays to exceed 64K
 - flat: no segments, 32-bit addresses, protected mode only (80386 and higher)

Program Skeleton

```
.model small
.stack 100H
.data
    ;declarations
.code
main proc
    ;code
main endp
    ;other procs
end main
```

- Select a memory model
- Define the stack size
- Declare variables
- Write code
 - organize into procedures
- Mark the end of the source file
 - optionally, define the entry point

```
A 100      ; BEGIN ASSEMBLY AT LOCATION 100H
MOV AX,5    ; FIRST PROGRAM STATEMENT
MOV AX,10
ADD AX,20
MOV [120],AX ; SUM IS AT LOCATION 0120H
INT 20      ; END PROGRAM
            ; PRESS ENTER TO END ASSEMBLY
R          ; DISPLAY THE REGISTER
T          ; TRACE ONE INSTRUCTION
T
T
G ; EXECUTE THE REST OF THE PROGRAM
Q          ; QUIT DEBUG AND RETURN TO DOS
```